一、git是什么

git是由Linux开发者Linus用了开发的分布式版本控制系统系统。

核心特点：

分布式架构：每个开发者拥有完整仓库副本，支持离线工作及快速分支合并。

分支与合并：支持创建并行开发路径（如功能分支、修复bug分支），完成后可合并至主线。

数据完整性：通过SHA-1散列算法校验文件及更改，确保数据无篡改。

高效性能：本地执行大部分操作，无需依赖远程服务器，适用于大型项目。

除了git，还有svn、cvs这样的版本控制系统，它们的区别在于一个是分布式一个是集中式。

集中式，就是指每次写代码，都需要从服务器中拉取一份，如果服务器故障，那么所有的数据就都丢失了，本机客户端仅保存本机的版本信息。就是说，集中式是把代码放在一个服务器上集中管理，所有的回滚操作都需要服务器的支持。

分布式，指每个人的电脑都是一台服务器，当从主仓库拉取一份代码时，你的电脑就会保存这份代码，可以在本地自由的对代码进行回滚、提交，当你想把自己的代码提交到主仓库时，只需要合并推送到主仓库就可以了，同时你可以把自己的代码新建一份仓库分享给其它人。

对于版本号，集中式是都有一个主版本号，所有的版本迭代都以这个版本号为主，而分布式因为每个客户端都是服务器，git没有固定的版本号，所以是有一个由哈希算法计算出的Id，用来回滚用的。同时也有一个master仓库，这个仓库是一切分支仓库的主仓库，我们可以推送提交到master并合并到主仓库上，主仓库的版本号会迭代一次，我们客户端上的git版本号无论迭代多少次，都跟master无关，只有合并时，master才会迭代一次。

二、git可以用来干什么

在我的经历中，git最多用在版本管理和合作开发。版本管理，就是可以保存你每次开发的历史版本（你提交了的话），如果你新功能的开发影响到了旧功能，或者产生了不好修复的bug，可以回退到上一个版本，这种优秀的版本管理可以极大的提升开发效率。不用说为了改回原来的代码，删删改改，最后导致所有的功能不可用。

还有一个，则是合作开发的问题，试想一下，如果大家在同一个文件夹里开发代码，他改改这边，你修修那边，改着改着突然发现他不小心/故意动了你代码的某个地方，导致你的代码全篇崩溃，含泪修复。但是有了git，你们可以聪明的各自从主仓库拉一份相同的代码下来，在各自的电脑上修改，然后提交推送。这时候就有人要问了，如果他推送的又把你的代码改了呢。诶，首先呢，按照开发规范，你们应该在主分支上新建自己的分支，拉取代码推送代码，都该在自己的分支上进行，这样无论他怎

么改，都影响不到你自己的分支。等到最后你们都开发完，确定他没有整什么幺蛾子，你们就可以把代码合并回到主分支。**可以把git的分支开发想象成一条直直的公路，你和你的合作伙伴是并行的两辆小轿车，你俩的目的地是完成项目的开发。而分支呢，就是这条大路上的两条岔路，你要去左边的服务区，他要去右边的风景区。等你俩完成各自的事情，在从岔路上走回大路，驶向最后的目的地**

三、git 常用命令

1.git config

初次使用git需要设置你的用户名以及邮箱，这将作为当前机器git的标识，如果你用它来下载远程仓库一些需要登录权限的仓库会要求登录，git默认使用配置邮箱以及用户名登入，但会要求你手动输入密码。通过git config命令则是用于管理我们的配置文件的命令

代码块

```
git config [<options>] [<key>] [<value>]
```

- options:指定配置层级（系统/全局/本地）
- key:要配置的键（例如user.name）
- value:该键对应的值

常见的配置项

1.用户身份配置（必配）

用户姓名：user.name

代码块

```
# 全局配置（推荐）
git config --global user.name "John Doe"
# 本地仓库单独配置（覆盖全局）
git config --local user.name "Project Maintainer"
```

用户邮箱：user.email

代码块

```
git config --global user.email "john.doe@example.com"
```

2.代理与网络配置

HTTPS代理：http.proxy

```
代码块
1   # 全局配置代理（HTTP/HTTPS）
2   git config --global http.proxy http://proxy.example.com:8080
3   # 取消代理
4   git config --global --unset http.proxy
```

Git协议代理：core.gitproxy

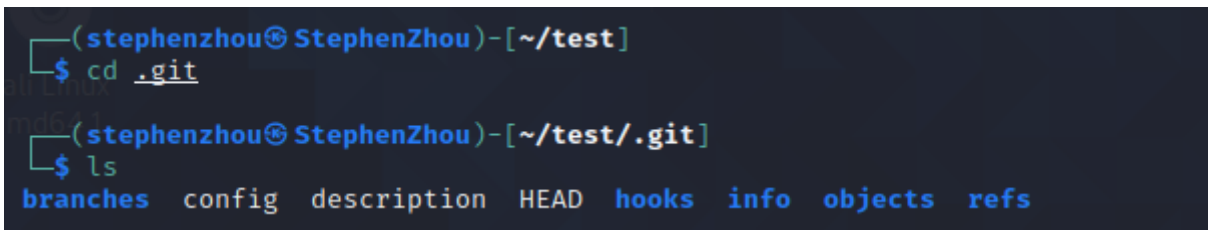
```
代码块
1   git config --global core.gitproxy "socks5://proxy.example.com:1080"
```

2.git init

初始化当前目录为仓库，初始化后会自动将当前仓库设置为master

首先创建一个空目录，进入目录，在该目录下的终端输入命令，初始化该仓库。

进入该目录，将看到因为命令存在的相关配置文件



3.git add 与 git commit

这两个命令常常配合在一起使用。add是将更改过的文件添加到缓存区，commit是将文件提交到本地仓库。

3.1 git add

假设，我们此时创建了一个文库test.go，使用add命令：

```
代码块
1   git add test.go
```

这时候的文件只是放在了缓存区，并没有提交到我们的仓库中。为什么不直接提交，而要先添加到缓冲区呢。我觉得这是一个效率上的问题。缓存有两种，一种是cache，用来解决两个模块之间数据存储的速度不匹配的问题，还有一种是buffer，用来防止小数据一点一点提交，先收集在一起，在整块提

交。我觉得这里的add的缓存区就像是一种buffer，因为代码的修改是频繁的，并且是多个文件的修改，所以我们一次性提交文件中所有修改的地方和所有被修改过的文件，可以很好的提高性能。

以下是从网络整理来的add的意义：

1. 版本控制的基石：
   - 精确控制：通过 `git add`，开发者可以精确控制哪些更改被包含在下一个提交中。这对于保持提交历史的清晰和逻辑性至关重要。
   - 避免混乱：如果没有 `git add`，所有更改将直接进入提交，这可能导致提交历史混乱，难以回溯和理解。
2. 协作的基础：
   - 团队协作：在团队环境中，`git add` 允许开发者选择性地提交更改，避免将不相关的更改混合在一起，从而减少冲突和混乱。
   - 代码审查：通过分阶段提交，代码审查变得更加容易，因为每个提交都对应一个具体的更改或功能。
3. 灵活性：
   - 撤销更改：使用 `git add` 后，如果开发者改变主意，可以使用 `git reset` 或 `git restore` 来撤销暂存区的更改。
   - bash

```
代码块
1   git reset HEAD file1.js   # 撤销暂存区的更改
```

4. 性能优化：

- 增量提交：通过 `git add`，开发者可以逐步提交更改，避免一次性提交大量更改，从而提高提交效率和性能。

3.2 git commit

commit就是把缓存区中的信息，提交到本地仓库中，提交时，要附带你提交的描述信息。

常用以下命令的形式：

```
代码块
1   git commit -m "add new file \"test.c\""
```

-m后是你简略的描述信息，如果没有-m，没写描述信息，会调用终端的注释编辑器让你输入描述信息，但是不建议使用，因为注释编辑器比较难用，不舒服。为了更好的团队合作，commit的提交信息

是有一定规范的，而不是想写写什么。在常用参数后，我会写一种常见的信息格式。

常用参数：

参数/选项	作用	示例命令
`-m "message"`	直接在命令行中指定提交信息	`git commit -m "Fix login bug"`
`-a`	自动暂存所有已跟踪文件的更改（不包括新文件），然后提交	`git commit -a -m "Update README"`
`--amend`	修改上一次提交（可用于修正提交信息或添加遗漏的更改）	`git commit --amend -m "Corrected commit message"`
`--no-verify`	跳过 pre-commit 钩子和 commit-msg 钩子	`git commit -m "Quick fix" --no-verify`
`--author="name <email>"`	指定提交作者（覆盖默认配置）	`git commit -m "Refactor code" --author="Alice <alice@example.com>"`
`--date="date"`	指定提交日期（通常用于修正历史记录）	`git commit -m "Initial commit" --date="2023-01-01"`
`--allow-empty`	允许创建空提交（无更改的提交，通常用于触发 CI/CD）	`git commit --allow-empty -m "Trigger build"`
`-v`	在提交信息编辑器中显示差异（verbose）	`git commit -v -m "Update documentation"`

提交规范：

提交信息一般包含三部分，类型，题目，描述。

- 类型（Type）：用来明确更改的形状
- 标题（Subject）：简明扼要地描述更改内容（不超过 50 个字符）。
- 正文（Body）（可选）：详细解释更改的原因和具体内容。

例如：

代码块

```
feat: 添加用户登录功能

实现了基于 JWT 的用户认证机制，支持邮箱和密码登录。
修复了登录状态管理中的潜在问题。
```

当然，平时也可以直接使用 类型：描述信息的格式

常用的类型：

- `feat`：新功能
- `fix`：修复 bug
- `docs`：文档更新
- `style`：代码格式（不影响功能）
- `refactor`：重构代码（既不修复 bug 也不添加新功能）
- `test`：添加或修改测试
- `chore`：构建过程或辅助工具的变动

git commit 会为我们生成40位的哈希值，用于作为id，并把刚刚用git add添加到提交缓存区里的文件提交到本地仓库中，便于我们回滚，至此，这个文件就已经添加到本地仓库中了，同时本地仓库也迭代了一个版本。

4.git log

log命令是查看日志的命令，可以查看改写的提交是否成功

引用CSDN（https://blog.csdn.net/bjbz_cxy/article/details/116703787）上的一个例子：

```
┌──(stephenzhou㉿StephenZhou)-[~/test]
└─$ git log
commit b9e3a0d708ee5a81ea5ff383c6dabe716eec8cf1 (HEAD → master)
Author: StephenZhou <stephenzhou@StephenZhou.www.malloc.com>
Date:   Tue Dec 29 12:15:13 2020 +0800

    test add new file "test.c"
```

第一行的commit是哈希算法算出的id，正如一开始所说，分布式是没有一个主版本号的，它们都是用id来做标志的，同时用master作为主仓库，其它的分支怎么迭代都不会影响到master

目前我们的仓库就是master，因为我们没有拉取分支是直接用git init创建的，就是master。

后面的head是指向的意思，表示这次提交到哪儿，head->master代表这次提交到master主仓库，如果是head->分支仓库则代表提交到分支仓库

Author是提交者是谁的意思，显示格式是：用户名 <邮箱>

Date的意思是提交时间，后面的+0800这个是格林尼治时间，代表当前是以哪儿的时间地作为基准，这是世界时间，用它来作为基数与当前所在地时差进行计算，包括地球自转等公式。

5.git reset

这个命令是用来回滚代码仓库的，一共有三种恢复的等级：

--soft：仅仅将头指针恢复，已经add的暂存区以及工作空间的所有东西都不变

--mixed：将头恢复掉，已经add的暂存区也会丢失，工作空间的代码是不变的

--hard：将一切都恢复，头变，add的暂存区也消失，代码什么的也恢复到以前的状态

回滚到历史版本示例：

代码块

```
1   git reset --hard 要回滚id
```

回滚当前仓库指向的版本：

代码块

```
1   git reset --hard HEAD^
```

^代表上一个版本的意思，HEAD代表当前仓库的指向，当前HEAD指向master，就代表回滚到master上一次提交的版本

6.git创建分支

6.2 git branch

git branch 用于查看、创建、删除分支、重命名分支，通过不同的命令参数实现相应的功能

常见用法：

查看分支：

代码块

```
1   git branch #只列出本地分支，并在当前分支前面使用*标记
```

代码块

```
1   git branch -r #该选项是只列出远程分支，本地分支不会显示
```

代码块

```
1   git branch -a #查看所有分支，包含本地分支及其对应的提交记录
```

创建分支：

以当前分支为起点，创建一个新分支【使用前提: 当前分支已有提交记录】

代码块

```
1   git branch dev #创建分支dev
```

注意：在任何分支下创建文件，若没有提交到仓库，那它在所有仓库下都是可见的，比如你在分支dev中创建了一个文件，没有使用git add和git commit提交，此时你切换到master，这个文件依旧存在的，因为你创建的文件在工作目录中，你切换仓库时git只会更新跟仓库有关的文件，无关的文件依然存放在工作区。

删除分支：

代码块

```
1   git branch -d <branch> #删除完全合并的分支
```

代码块

```
1   git branch -D <branch> #删除分支（即使未合并），本地分支已有新的提交记录，但没推送到远程仓库，此时想要删除分支，必须使用 -D
```

代码块

```
1   #删除远程分支
2   # 普通写法
3   git push origin -d <branch>/git push origin --delete [branchname]
4   # 简写形式
5   git push origin :<branch>
```

移动/重命名分支：

代码块

```
1   git branch -m <old_name> <new_name> #重命名分支。
2   git branch -M <old_name> <new_name> #强制重命名分支，即使目标分支已存在。
```

合并分支：

代码块

```
1   git branch --merge <branch_name> #列出已经合并到当前分支的分支。
```

6.2 git checkout

git checkout 命令是 Git 中用于切换分支、恢复文件内容和创建新分支的主要命令。它是 Git 操作中非常常见的一部分，常被用作切换分支、恢复文件、创建新分支

git命令的基本用法：

```
代码块
1   git checkout [选项] <分支名或提交哈希> [文件路径...]
```

常见用法：

切换分支：

```
代码块
1   git checkout <branch-name>
```

创建并切换到新分支：

```
代码块
1   git checkout -b <new-branch-name> #-b选项可以在切换的同时创建一个新的分支
```

恢复文件的指定版本：

```
代码块
1   git checkout <commit-hash> -- <file-path> #可以恢复某个文件到指定提交时的状态。
```

切换到远程分支：

```
代码块
1   git checkout -t origin/<branch-name>#git checkout -t origin/<branch-name>
```

6.3 总结

git branch更多用来对分支进行管理，比如创建和删除，git checkout更多用来查看分支

分支的使用建议：

频繁沟通：与团队成员定期沟通分支的状态和进度，避免不必要的冲突和重复工作。

及时合并：定期将分支合并到主分支，减少合并时的冲突。

使用分支策略：根据项目需求选择合适的分支策略，如 Git Flow 或 GitHub Flow。

清理无用分支：定期检查并删除不再需要的分支，保持仓库的整洁。

7.git合并分支

当新建分支的工作做完之后，想要把分支提交到master，需要先切换到master仓库，并执行git merge 分支名 命令，将分支合并。

分支的重要性：

在开发中，所有的操作都建议大家拉取分支后执行。因为在团队开发中，master只有一个，大家都可以从master拉取代码，拉取master后创建分支，分支名改为自己要做的操作（与git commit的提交信息同理，如：feat/某某文件）。一般情况下，我们都是拉取master后，想要修改功能或者添加功能，都是创建分支，在分支里修改不影响master，如果修改错了代码或者误删之类的，在从master上拉取一份就可以了。

四、工作区与缓存区

工作区：指当前工作的目录

缓存区：存放git add命令提交的文件描述信息，位于.git目录下的index文件中

HEAD文件就是指向当前的仓库，最后使用git commit提交时会提交到当前仓库中，当前的工作区也就成为了最新一次提交的仓库版本。

五、git的基本组成框架

git的基本组成框架为Workspace、Index / Stage、Repository、Remote

Workspace：开发者工作区，是当前写代码的目录，一般保持的是最新仓库代码

Index / Stage：缓存区，最早叫Stage，现在新版本已经改成index，位于.git目录中，它用来存放临时动作，比如我们做了git add或者git rm，都是把文件提交到缓存区，这是可以撤销的，然后在通过git commit将缓存区的内容提交到本地仓库。

Repository：仓库区，是仓库代码，你所有的提交都在这里，git会保存好每一个历史版本，存放在仓库区，它可以是服务端的也可以是本地的，因为在分布式中，任何人都可以是主仓库。

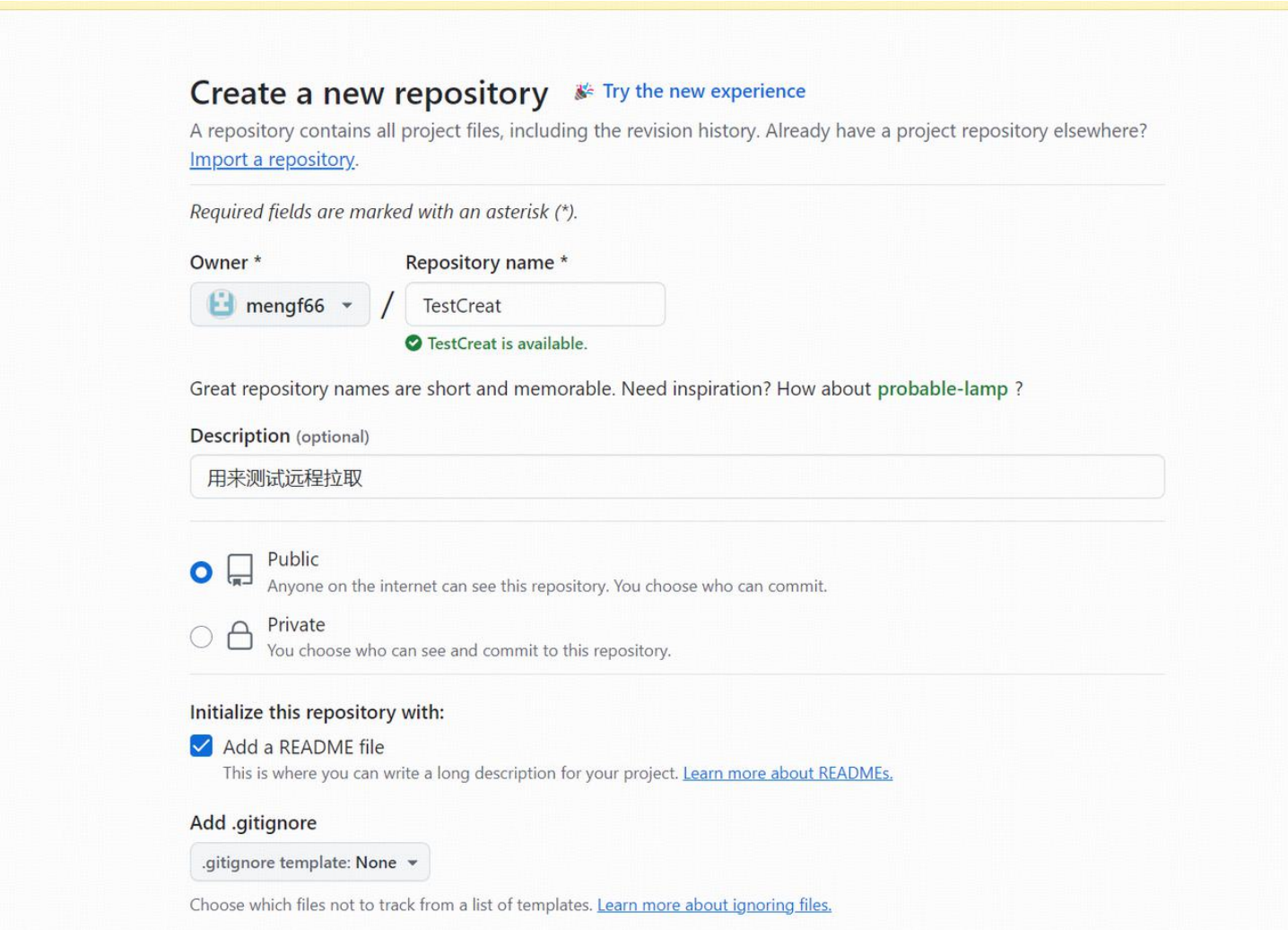
Remote：远程仓库，只能是别的电脑上的仓库，即服务器仓库。

六、git使用流程

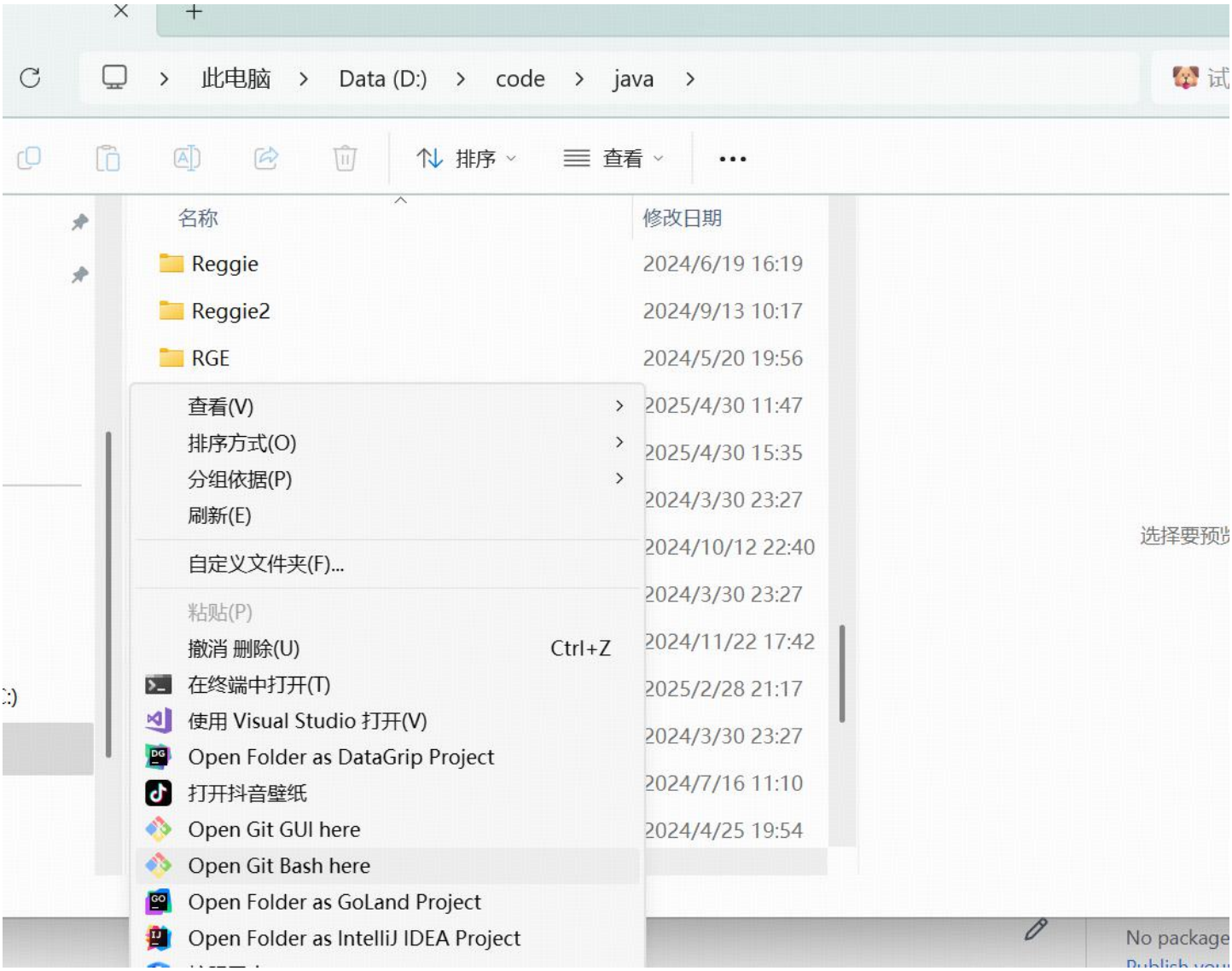
1.个人使用

首先根据SSH详解中的步骤，远程连接好github

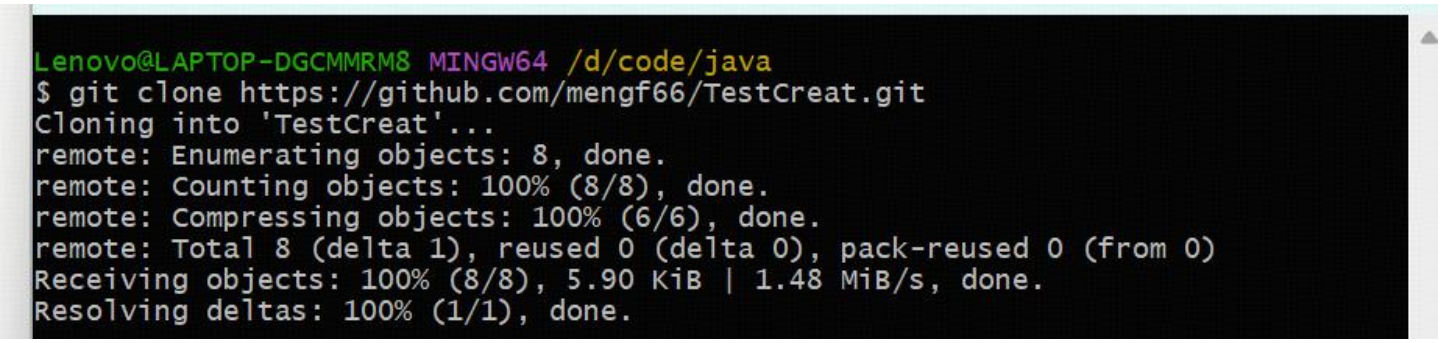
接下来，在github上创建你的个人仓库。



在个人电脑上的代码文件夹中打开git bash终端界面，拉取远程仓库到本地：



使用git clone命令，拉取对应仓库（**注意：这里我偷懒使用了http连接，建议大家选择SSH连接**）



代码块

```
git clone 仓库路径
```

命令执行完成，会在当前目录下自动创建对应文件夹。

使用git checkout命令，切换到对应分支

```
Lenovo@LAPTOP-DGCMMRM8 MINGW64 /d/code/java/TestCreat (main)
$ git checkout feat/test-pull
```

在分支下创建文件，使用git add ，git commit提交

```
Lenovo@LAPTOP-DGCMMRM8 MINGW64 /d/code/java/TestCreat (feat/test-pull)
$ mkdir test
```

```
Lenovo@LAPTOP-DGCMMRM8 MINGW64 /d/code/java/TestCreat (feat/test-pull)
$ touch test/hello.java
```

```
Lenovo@LAPTOP-DGCMMRM8 MINGW64 /d/code/java/TestCreat (feat/test-pull)
$ git add test/hello.java
```

```
Lenovo@LAPTOP-DGCMMRM8 MINGW64 /d/code/java/TestCreat (feat/test-pull)
$  git commit -m "feat/创建了新文件hello"
[feat/test-pull 608ba1e] feat/创建了新文件hello
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 test/hello.java
```

代码块

```
git checkout
mkdir test
touch test/hello.java
git add test/hello.java
git commit -m "feat/创建了新文件hello"
```

列出远程分支名，使用git push命令提交到对应的远程分支上

```
Lenovo@LAPTOP-DGCMMRM8 MINGW64 /d/code/java/TestCreat (feat/test-pull)
$ git branch -r
  origin/HEAD -> origin/main
  origin/feat/test-pull
  origin/main
```

- 若推送本地分支 `feat/test-pull` 到远程同名分支，应简写为：

bash　　取消自动换行 | 复制

```bash
git push -u origin feat/test-pull
```

- 若需显式指定远程分支名称（如远程分支名不同），则用 `:` 分隔：

bash　　取消自动换行 | 复制

```bash
git push -u origin feat/test-pull:feat/test-pull
```

```
Lenovo@LAPTOP-DGCMMRM8 MINGW64 /d/code/java/TestCreat (feat/test-pull)
$ git push -u origin feat/test-pull:feat/test-pull
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 28 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 347 bytes | 173.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:mengf66/TestCreat.git
   645a058..608ba1e  feat/test-pull -> feat/test-pull
branch 'feat/test-pull' set up to track 'origin/feat/test-pull'.
```

代码块

```
git branch -r
git push -u origin feat/test-pull
#或者
git push -u origin feat/test-pull:feat/test-pull
```

最后，将分支合并回主分支

git强制合并分支：--allow-unrelated-histories

当我们在使用两个不同的分支时或此分支不是从原生仓库中分支出来的，想要合并不符合GIT规则，所以会弹出：fatal: refusing to merge unrelated histories 的错误，比如当我们在本地开发好了，但是并没有在一开始关联远程仓库，若想提交就会出现这样的错误，我们先拉取下来以后合并分支在后面加上这条语句就可以了

代码块

```
git merge main --allow-unrelated-histories
```

注意，这里只是合并了本地分支，远程分支需要发送请求，仓库管理人同意后，才可以合并分支。

2.团队合作

大致流程都一样，创建分支，拉取仓库，推送代码，在主仓库发送合并请求，待请求通过合并入主分支。

团队合作的介绍，主要是在一些多人开发的问题上：

1.在你修改代码时，有人更改并推送了主分支代码，你如何解决代码冲突

代码块

```
# 1. 创建功能分支（基于最新 develop）
git checkout -b feature/login develop

# 2. 本地开发（高频小步提交）
git add .
git commit -m "feat(login): 实现 OAuth2 认证逻辑"

# 3. 定期同步主分支变更（防冲突）
git fetch origin
git rebase origin/develop  # 保持线性历史

# 4. 推送远程（首次推送需关联分支）
git push -u origin feature/login
```

2.分支命名不规范，导致的冲突与错误

- **强制分支规范**

markdown　取消自动换行 | 复制

```markdown
# 分支命名规则
- feature/* : 新功能（如 feature/user-auth）
- bugfix/*  : 缺陷修复（如 bugfix/login-fail）
- hotfix/*  : 生产环境紧急修复
- release/* : 版本发布分支
```

- **定期清理分支**

bash　取消自动换行 | 复制

```bash
# 删除已合并的远程分支
git push origin --delete feature/old-feature
```

3.频繁的合并冲突

- **高频小步提交 + 及时同步**

bash 取消自动换行 | 复制

```bash
# 开发中每完成小功能就提交
git add .
git commit -m "feat: 完成登录表单校验"

# 每日同步最新代码
git pull origin develop --rebase  # 使用 rebase 保持线性历史
```

- **使用 `.gitattributes` 解决换行符冲突**

ini 取消自动换行 | 复制

```ini
# .gitattributes
* text=auto eol=lf
```

4.大文件管理问题

- **使用 Git LFS**

bash 取消自动换行 | 复制

```bash
# 安装 Git LFS
git lfs install

# 追踪大文件（如 .psd, .zip）
git lfs track "*.psd"
git lfs track "*.zip"
```

- **添加 `.gitignore`**

gitignore 取消自动换行 | 复制

```gitignore
# 忽略依赖目录
node_modules/
.idea/
```

总结：

团队协作的关键是：

- 明确的分支策略与规范
- 自动化检查 + 人工审查结合
- 高频同步 + 小步提交
- 权限分级 + 安全保护

引用文章（一些很好的文章，建议大家去看看）

git常用命令详解（CSDN）

git checkout命令详解

git push命令

git branch命令详解

git连接错误解决方案